Assignment 4

Task 1 LL(1) Property

Consider the following EBNF grammar, which is part of a complete grammar.

```
FunctionDecl = “def” ident “(“ [ ArgList ] “)” “->” Type Block .
ArgList = ArgList “,” Arg | Arg .
Arg = ident “:” Type .
Type = “num” | “char” | “bit” .
Block = “:” InstructionList | “:” .
InstructionList = { Instruction "," } Instruction .
Instruction = ident | ident “<-” Expr .
```

(a) Explain why this grammar is not LL(1).

A **LL(1)**:

- can be analyzed from **L**eft to right
- with **L**eft-canonical derivations (leftmost NTS is derived first)
- and **1** lookahead symbol

Definition

1. A grammar is LL(1) if all its productions are LL(1).
2. A production is LL(1) if for all its alternatives $\alpha 1 \mid \alpha 2 \mid \ldots \mid \alpha$ n the following holds: $\text{FIRST}(\alpha i) \cap \text{FIRST}(\alpha j) = \{\}$.
3. The terminal start symbols of all alternatives in a production must be pairwise disjoint.
4. The parser must always be able to decide on one of the alternatives based on the lookahead token.

1. **Left Recursion in** `ArgList`:
   The production `ArgList = ArgList "," Arg | Arg` is **left-recursive** because the nonterminal `ArgList` appears as the first symbol of its own first alternative. Left recursion is a fundamental $LL(1)$ conflict that causes recursive descent parsers to enter infinite loops.
2. **Common Prefixes in** `Instruction`:
   The production `Instruction = ident | ident "<-" Expr` violates the $LL(1)$ requirement that terminal start symbols of all alternatives must be pairwise disjoint. Since the intersection is not empty, the parser cannot decide which alternative to choose based on the lookahead `ident`. Same is true for `Block`.
3. **Hidden LL(1) Conflict in** `InstructionList`:
   The production `InstructionList = { Instruction "," } Instruction` contains a hidden conflict. Because `Instruction` starts with `ident`, the parser cannot distinguish between starting another iteration of the loop or recognizing the final mandatory `Instruction`.

(b) Rewrite the grammar such that it is LL(1).

To transform the grammar into $LL(1)$, we apply **Factorization** to remove common prefixes and replace recursion with **Iteration**.

How to eliminate left recursion

How to remove left recursion

Left recursion is always an LL(1) conflict

For example

```
IdentList = ident | IdentList "," ident.
```

... both alternatives start with *ident*

can always be replaced by iteration

```
IdentList = ident {"," ident}.
```

Problem: ArgList = Arglist "," Arg | Arg
Resolution: ArgList = Arg { "," Arg}

Left Factorization

Left factoring is a grammar transformation used when two or more alternatives start with the same prefix, and the parser can't decide which one to pick using only one look-ahead token.

Inline *Designator* in *Statement*

```
Statement  =  ident {"." ident} "=" Expr ";"
           |  ident "(" [ActualParameters] ")" ";".
```

then factorize

```
Statement  =  ident  (  {"." ident} "=" Expr ";"
                     |  "(" [ActualParameters] ")" ";"
                     ).
```

Problem: Block = ":" InstructionList | ":".
Resolution: Block = ":" [InstructionList].

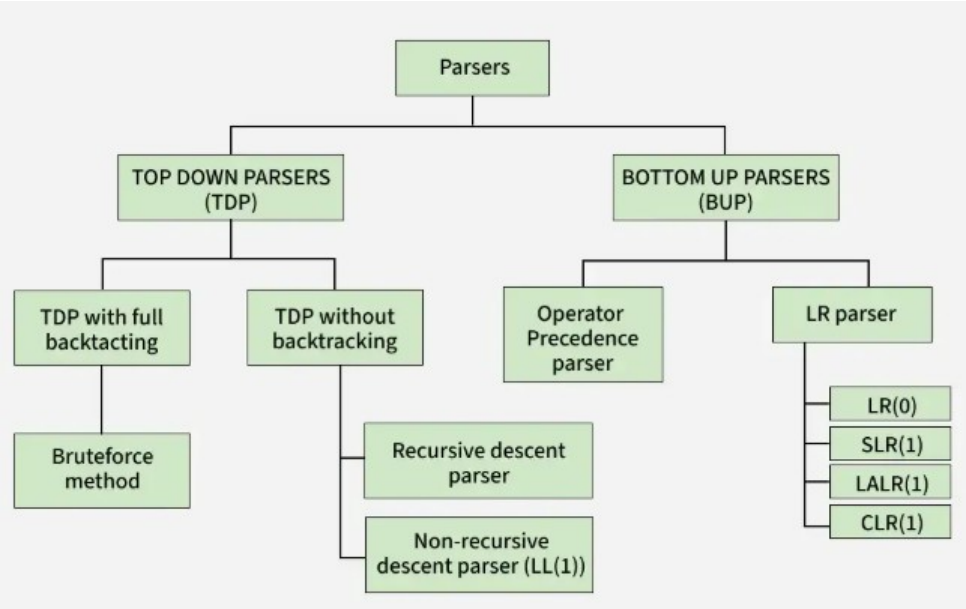
Problem: Instruction = ident | ident "<−" Expr.
Resolution: Instruction = ident ["<−" Expr].

Hidden LL(1) conflict

Problem: InstructionList = { Instruction ","} Instruction.
FIRST({ Instruction ","}) ∩ FIRST(Instruction) ≠ {}
Resolution: Instructionlist = Instruction { "," Instruction }.
Check: FIRST(Instruction) ∩ FOLLOW(InstructionList) = {}

Nonterminal	Production Rule
FunctionDecl	"def" ident "(" [ArgList] ")" "->" Type Block.
ArgList	Arg { "," Arg }.
Arg	ident ":" Type.
Type	"num" | "char" | "bit".
Block	":" [InstructionList].
InstructionList	Instruction { "," Instruction }.
Instruction	ident ["<-" Expr].

Table 1: Revised LL(1) Grammar using EBNF notation.



Task 2 Error handling with general anchors

For the following grammar, write Java parser methods with error handling using general anchors and successor sets (BitSet sux). You do not need to implement an executable parser, i.e., untested parser methods are sufficient for this task.

```
S = Q | N P.
Q = n | i k .
N = "q" { Q } .
P = [ p ] e .
```

```java
/**
 * S = Q | N P
 * First(Q) = {n, i}, First(N) = {q}
 */
static void S(BitSet sux) {
    BitSet firstQ = new BitSet().set(n).set(i);
    BitSet firstN = new BitSet().set(q);

    if (firstQ.get(sym)) {
        Q(sux);
    } else if (firstN.get(sym)) {
        // Successors of N are First(P) + sux(S)
        BitSet firstP = new BitSet().set(p).set(e);
        N(new BitSet().union(firstP).union(sux));
        P(sux);
    } else {
        error("n, i or q expected", firstQ.union(firstN).union(sux));
```

```
    }
}

/**
 * Q = n | i k
 */
static void Q(BitSet sux) {
    if (sym == n) {
        scan();
    } else if (sym == i) {
        scan();
        check(k, sux);
    } else {
        error("n or i expected", new BitSet().set(n).set(i).union(sux));
    }
}

/**
 * N = "q" { Q }
 */
static void N(BitSet sux) {
    check(q, new BitSet().set(n).set(i).union(sux));

    BitSet firstQ = new BitSet().set(n).set(i);
    while (true) {
        if (firstQ.get(sym)) {
            Q(new BitSet().union(firstQ).union(sux));
        } else if (sux.get(sym) || sym == eof) {
            break; // Valid successor reached
        } else {
            // Error handling inside the loop
            error("Start of Q (n or i) expected", firstQ.union(sux));
            if (!firstQ.get(sym)) break;
        }
    }
}

/**
 * P = [ p ] e
 */
static void P(BitSet sux) {
    if (sym == p) {
        scan();
    } else if (sym != e && !sux.get(sym)) {
        error("p or e expected", new BitSet().set(p).set(e).union(sux));
    }
    check(e, sux);
}
```

Task 3 Parser implementation

Write an executable Java program containing scanner and parser for this grammar. Test your implementation using multiple valid and invalid input files.

```
Object = "{" [ Pair { "," Pair } ] "}".
Pair = string ":" Value.
Value = string | "null" | Object | Array.
Array = "[" [ Value { "," Value } ] "]".
```

Error Handling Strategy (Specific Anchors)

Following the strategy in reference 3.11, we implement synchronization points at the start of repeating elements (like `Pair` and `Value`). If an error occurs, the parser skips tokens until it finds a "synchronization symbol" (e.g., a comma, a closing brace, or the start of the next element) to prevent infinite loops and cascading errors.

Implementation

1) JsonParser.java is a standalone Java program implementing both the Scanner and the Recursive-Descent Parser.

- The **Scanner** handles reading characters and extracting tokens (LBRACE , RBRACE , LBRACK , RBRACK , COMMA , COLON , STRING , NULL-LIT , EOF). It skips whitespace and reports errors for invalid characters or unclosed strings.
- **Parser**: implements the grammer using recursive descent with the functions (parseObject() , parsePair() , parseValue() , and parseArray()).

.

valid1.json: a simple valid JSON configuration testing basic pairs and null.
valid2.json: a complex JSON object testing nested objects, arrays and empty objects/arrays.
invalid1.json: Tests parsing an unquoted string key (which shouldn't be possible in our grammar).
invalid2.json: Testd a missing comma between pairs.
invalid3.json: Tests a trailing comma before the closing brace.

Execution of the program

```
>java JsonParser valid1.json
File 'valid1.json' parsed successfully!

>java JsonParser valid2.json
File 'valid2.json' parsed successfully!

>java JsonParser invalid1.json
Syntax Error at line 3, col 9: Expected STRING, but found ERROR

>java JsonParser invalid2.json
Syntax Error at line 3, col 9: Expected RBRACE, but found STRING

>java JsonParser invalid3.json
Syntax Error at line 3, col 9: Expected STRING, but found RBRACE
```

Task 4 Error handling with specific anchors

Design and implement an error handling strategy using specific anchors by extending your parser of Task 3. Demonstrate using additional invalid input files that your parser can detect and recover from multiple errors in an input file.